

Knowledge Representation & Reasoning in AI

Part 1

Non-Monotonic
Reasoning

Part 2

Commonsense
Reasoning

Part 3

Temporal
Reasoning

Part 4

Knowledge
Graphs

Part 5

Neuro-Symbolic
AI

TOPIC 01

Non-Monotonic Reasoning

When learning something new means taking back what you believed before.

Your GPS Changes Its Mind

Imagine you are driving from Lahore to Islamabad. Your GPS says:

"Take Motorway M-2 — this is the fastest route."

Thirty minutes later, the GPS suddenly says:

"Accident detected ahead. Heavy traffic congestion. Take an alternative route."

Did the GPS make a mistake the first time? No.

It made the best decision possible with the information available at that moment. When new information arrived, it revised its conclusion. This ability — to take back an earlier, reasonable conclusion when new facts arrive — is called ***non-monotonic reasoning***, and it is something humans (and good AI systems) do every single day.

What Does "Non-Monotonic" Actually Mean?

In everyday math, if you know that $2 + 2 = 4$, no new fact you ever learn will change that. Mathematical logic is built so that the set of true conclusions only ever **grows** — it never shrinks. We call this property *monotonicity*: once something is proven true, adding more facts can never make it false again.

$\text{Conclusions}(K1) \subseteq \text{Conclusions}(K2)$
"Adding information can only create new conclusions.
It can never invalidate old conclusions."

But the real world doesn't work that way. Someone tells you "**Tweety is a bird.**" Your natural reaction: "**Tweety can fly**" — most birds fly. Now they add: "**Tweety is a penguin.**" Suddenly your earlier conclusion collapses — Tweety cannot fly after all. You didn't make a mistake; the new fact simply forced you to **withdraw** an earlier, reasonable belief.

Non-Monotonic Reasoning — Formal Definition

DEFINITION

Non-monotonic reasoning is a style of reasoning in which adding new information to a knowledge base can cause previously derived conclusions to be retracted, not just added to. It models how humans reason under incomplete information — we jump to reasonable conclusions and revise them later if better evidence appears.

Key idea: classical logic only grows its conclusions. Non-monotonic reasoning can shrink them too.

The Real World Is Full of Exceptions

Most human knowledge is not absolute. Consider three ordinary statements and how easily each one breaks:

"Always True" Statement	Common Exceptions
Birds fly.	Penguins, ostriches, emus, injured birds
Students attend classes.	Sick students, students on leave, students at a conference
Roads are open.	Construction, floods, accidents

The world contains exceptions everywhere. Therefore, AI systems need a reasoning mechanism capable of handling exceptions gracefully — without grinding to a halt or producing a contradiction every time an exception appears.

WHY CLASSICAL LOGIC STRUGGLES HERE

Suppose a knowledge base contains $\text{Bird}(\text{Tweety})$ and the rule $\text{Bird}(X) \rightarrow \text{Fly}(X)$. Classical logic derives $\text{Fly}(\text{Tweety})$. Now we add $\text{Penguin}(\text{Tweety})$ and $\text{Penguin}(X) \rightarrow \neg \text{Fly}(X)$. Classical (monotonic) logic has no clean mechanism to "take back" the first conclusion — it simply ends up holding two contradictory statements at once. Non-monotonic reasoning exists specifically to resolve this.

Default Reasoning

Default reasoning is the most important and most natural form of non-monotonic reasoning. Humans use it constantly: when you see a bird, you assume it can fly without first checking thousands of possible exceptions. You use the default assumption and move on, ready to revise it if contradicted.

DEFAULT RULE (general form):

IF condition is true AND there is no evidence against it
THEN assume the conclusion.

Bird(X) $\Rightarrow$ Fly(X) "If X is a bird and nothing contradicts flying, assume X flies."

Proposed formally by **Raymond Reiter**, default logic adds these special defeasible rules on top of ordinary logical rules. The phrase "*unless stated otherwise*" captures the idea perfectly.

WHY DEFAULT REASONING IS POWERFUL

Without default reasoning, an AI system would need complete knowledge before making any decision — an impossible standard. Imagine a robot that refuses to move because "I don't know everything about this room." Humans don't operate this way; we make reasonable assumptions and adjust later. Default reasoning lets machines do the same.

The Tweety Triangle: Belief Formed, Then Withdrawn

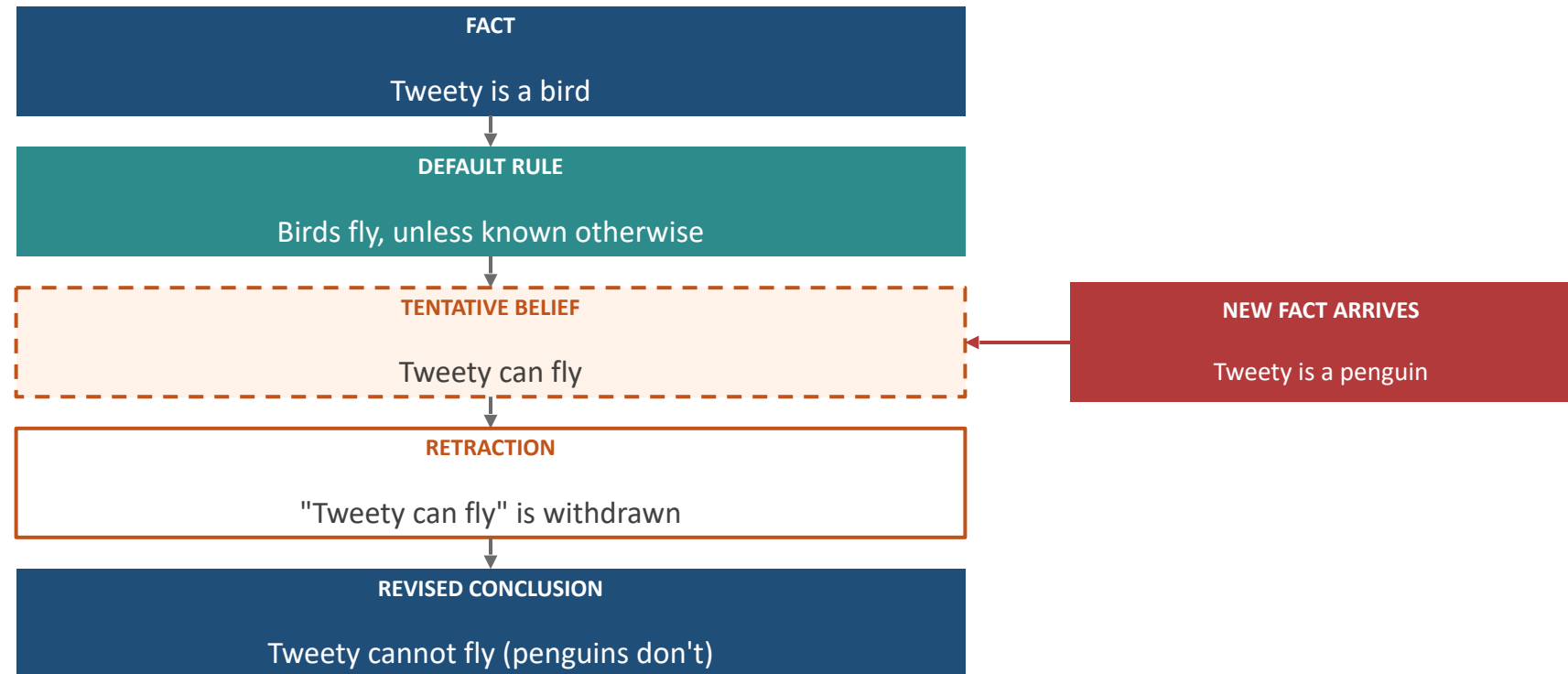


Figure 1.1 — A default belief is formed, then withdrawn the moment more specific information arrives.

IMPORTANT PRINCIPLE

Specificity beats generality. A more specific rule ("penguins don't fly") always overrides a more general default rule ("birds fly"). This is the Specificity Preference Principle, and it resolves most conflicts automatically.

Closed-World Assumption (CWA)

DEFINITION

Under the Closed-World Assumption, if a fact is not explicitly known to be true, the system assumes it is false. This is how most ordinary databases work.

Example: a student database lists Ali, Ahmed, and Sara. If you ask "Is Bilal a student?" and Bilal is not in the table, the database concludes Bilal is **not** a student — not "unknown," but definitively false. This works because the database is assumed to be a **complete** record of all students.

A second major tool in non-monotonic reasoning concerns how a system treats missing information — the next slide compares it directly with its counterpart, the Open-World Assumption.

Open-World Assumption (OWA) vs. CWA

DEFINITION

Under the Open-World Assumption, if a fact is not known to be true, the system does not assume it is false — it simply remains unknown. This is used in semantic web systems and knowledge graphs.

Example: a knowledge graph states "Ali studies at UMT." If you ask "Does Ali own a car?" and no such fact exists, OWA answers "**unknown**" — because the graph never claimed to be a complete record of everything true about Ali.

Feature	Closed-World (CWA)	Open-World (OWA)
Missing fact treated as	False	Unknown
Typical use case	Databases	Knowledge Graphs, Semantic Web
Assumes knowledge base is	Complete	Possibly incomplete
Risk if assumption wrong	False negatives (wrongly says "no")	Cannot confirm or deny — less decisive

Belief Revision

Belief revision is the general process of modifying beliefs when new information arrives — it is the broader principle that default reasoning, CWA, and truth maintenance all serve.

```
Knowledge → Inference → Conclusion → New Evidence  
→ Belief Revision → Updated Conclusion
```

Example: your initial belief is "the weather is sunny," so you decide to go for a picnic. A storm warning is issued. Your belief is revised to "bad weather is expected," and your decision changes accordingly. This cycle — belief, evidence, revision, updated belief — is the heart of intelligent decision-making under uncertainty.

Truth Maintenance Systems (TMS)

As a knowledge base grows, contradictions inevitably appear. A **Truth Maintenance System** keeps track of why each belief was added — which facts justify it — so that when a supporting fact disappears or is contradicted, the system can automatically retract every conclusion that depended on it, like a chain reaction of "un-believing."

Beliefs	Justifications	Dependencies
Things currently considered true	The reasons supporting each belief	Relationships between beliefs, so removing one can cascade to others

BUILDING ANALOGY

Think of beliefs as floors of a building, and justifications as the pillars holding each floor up. If a pillar (a supporting fact) is removed, the floor it supports collapses. Likewise, if the evidence behind a conclusion disappears, the conclusion itself must fall.

TMS Walkthrough — Road A Is Best, Until It Isn't

Belief: "Road A is best." Supported by: no traffic and shortest distance. Later: heavy traffic is reported on Road A.

- 1 It identifies that "no traffic" was one of the two justifications for "Road A is best."
- 2 Since that justification is now false, the TMS automatically marks "Road A is best" as unsupported and retracts it — without needing to re-examine the entire knowledge base.
- 3 The system updates its recommendation, e.g., to "Road B is best," based on whatever beliefs remain justified.

Real-World Applications of Non-Monotonic Reasoning

Domain	Initial Belief	New Information	Revised Conclusion
Medical Diagnosis	Flu	Lab test detects COVID	Diagnosis changes to COVID
Self-Driving Cars	Road is clear	Pedestrian detected by sensor	Brake immediately
Robotics	Door is open	Camera shows door closed	Plan is revised
Expert / Loan Systems	Customer eligible	Credit default history found	Decision reversed

The Classic Bird Problem

PROBLEM

You are told the following facts and rules, in this exact order:

1. Rule: "Birds typically fly, unless there is information that they cannot."
2. Fact: "Polly is a bird."
3. Fact (added later): "Polly has a broken wing."

Question: *Can Polly fly? What should the system believe after each new fact?*

Think it through step by step — what does the system believe after fact 2? What changes after fact 3?

PAUSE HERE — work out your answer before the next slide

The Classic Bird Problem — Step-by-Step

- 1 After Fact 1 ("Polly is a bird") is added, the default rule fires because nothing contradicts it yet. The system tentatively concludes: Polly can fly.
- 2 When Fact 2 ("Polly has a broken wing") is added, the system checks: does this new fact conflict with the default conclusion? Yes — a broken wing directly blocks flight.
- 3 The default rule's condition ("unless there is information that they cannot") is now satisfied by the broken-wing fact, so the rule no longer applies. The earlier conclusion is retracted.
- 4 Final belief: Polly cannot fly. No contradiction exists in the system, because the tentative belief was withdrawn rather than fought against.

WHY THIS SOLUTION WORKS

This example shows the heart of non-monotonic reasoning: the system never "knew" for certain that Polly could fly — it only assumed so because no counter-evidence existed at the time. The moment more specific, conflicting evidence (the broken wing) appeared, the assumption was dropped without any logical error. This mirrors how a doctor revises a diagnosis when a new test result comes in.

Closed-World Assumption in a Library System

PROBLEM

A library database contains book loan records. It lists:

"Ali borrowed Book A." "Sara borrowed Book B."

There is no record anywhere stating that Ahmed borrowed any book.

Question: *Using the Closed-World Assumption, what does the system conclude about whether Ahmed has borrowed Book C?*

PAUSE HERE — work out your answer before the next slide

Closed-World Assumption in a Library System

- 1 Identify what is explicitly stated in the database: only two loan facts exist ($\text{Ali} \rightarrow \text{Book A}$, $\text{Sara} \rightarrow \text{Book B}$). Nothing about Ahmed appears anywhere.
- 2 Apply the Closed-World Assumption: "If a fact is not present in the knowledge base, treat it as false." The database is assumed to be a complete record of all loans.
- 3 Since "Ahmed borrowed Book C" does not appear anywhere in the records, the system concludes this statement is false — Ahmed has not borrowed Book C.
- 4 Contrast with an Open-World Assumption system (like the public internet), which would instead answer "unknown" rather than "false," because it cannot assume its records are complete.

WHY THIS SOLUTION WORKS

Most everyday databases — banking, student records, library systems — silently rely on the Closed-World Assumption every day. It is a form of non-monotonic reasoning because if a new loan record for Ahmed is added tomorrow, the earlier conclusion is instantly retracted. The assumption only works because the domain is genuinely expected to be complete — this is why CWA fails badly on open domains like the web.

Conflicting Defaults — The Nixon Diamond

PROBLEM

Consider two default rules that both apply to the same individual:

Rule A: "Quakers are typically pacifists (against war)."

Rule B: "Republicans are typically not pacifists."

Fact: "Richard is both a Quaker and a Republican."

Question: *Should the system conclude Richard is a pacifist, not a pacifist, or something else? This famous puzzle is called the "Nixon Diamond."*

PAUSE HERE — work out your answer before the next slide

Conflicting Defaults — The Nixon Diamond

- 1 Check Rule A: Richard is a Quaker → default suggests pacifist.
- 2 Check Rule B: Richard is a Republican → default suggests not pacifist.
- 3 Check the Specificity Principle: is one rule more specific than the other? No — "Quaker" and "Republican" are independent categories; neither is a sub-case of the other.
- 4 Because neither default is more specific, the two conclusions are in genuine, unresolved conflict. A well-designed system reports: "The status is undetermined — both defaults are equally justified, and they cannot both hold."

WHY THIS SOLUTION WORKS

This example tests whether a reasoning system can recognize the limits of default reasoning. A poorly designed system might pick a conclusion at random — dangerous in real applications such as medical or legal AI. The correct, honest answer is to flag the conflict and ask for more information rather than silently guessing.

TOPIC 02

Commonsense Reasoning in AI Systems

Teaching machines the "obvious" things humans never think twice about.

What Is Commonsense Reasoning?

Consider this statement: "Ahmed dropped a glass cup onto a concrete floor." Even without any extra information, you instantly infer several things: the cup may break, pieces may scatter, someone might get injured, and the cup may no longer be usable. **Notice that none of this was explicitly stated** — yet every human understands it immediately. This is because humans possess commonsense knowledge: one of the greatest strengths of human intelligence, and historically one of the greatest weaknesses of AI.

DEFINITION

Commonsense reasoning is the ability of an intelligent system to use general world knowledge to make reasonable assumptions and draw conclusions in situations where information is incomplete.

Humans begin learning commonsense from birth. A child quickly learns that hot objects cause pain, dropped toys fall, people smile when happy, and food satisfies hunger. After years of interaction, billions of such small observations form a rich, implicit knowledge base — humans rarely need explicit rules; they simply "know."

Why Is This So Difficult for AI?

The challenge is not that commonsense is logically complex — it is that commonsense *appears* simple while hiding enormous amounts of implicit knowledge. Consider: "Ali entered a restaurant." Humans automatically infer that Ali is probably hungry, that there will be tables, that someone may take his order, and that payment will eventually occur. A computer sees only words; without additional knowledge, none of these conclusions are reachable.

■ Everywhere but never written down

Nobody writes an encyclopedia entry for "water is wet" — yet AI needs millions of such facts.

■ Requires combining many small facts at once

Understanding "She left her umbrella because it stopped raining" requires knowing what umbrellas are for, that rain stops, and why protection is no longer needed.

■ Full of exceptions

Closely related to the non-monotonic reasoning covered in Topic 1.

THE KNOWLEDGE ACQUISITION BOTTLENECK

Imagine manually teaching an AI everything a child knows: ice is cold, cars need fuel, doors can be opened, rain makes surfaces wet — then multiplying this by billions of real-world situations. This scale problem has challenged AI researchers for over fifty years.

Logical Reasoning vs. Commonsense Reasoning

	Logical Reasoning	Commonsense Reasoning
Style	Follows strict rules, guaranteed conclusions	Deals with uncertainty, defeasible conclusions
Example	"All humans are mortal; Socrates is human" → "Socrates is mortal" (certain)	"Ali entered a restaurant" → "Ali wants food" (usually true, not certain)

Commonsense reasoning is therefore *probabilistic and defeasible* rather than absolute — another reason it connects so closely to the non-monotonic reasoning of Topic 1. A conclusion that is "usually true" can always be overturned by a more specific fact, just as Tweety's ability to fly was overturned by learning he was a penguin.

Five Types of Commonsense Knowledge

Type	What It Covers	Example Inference
Physical	Objects and the natural world	"Sara threw a glass bottle onto the road" → it may break
Spatial	Location and containment	"Keys are in the drawer; drawer is in the desk" → keys are in the desk
Temporal	Order and timing of events	"Ali graduated in 2020" → he was a student before 2020
Social	Human interactions and roles	"Ahmed visited a doctor" → he may have a health concern
Psychological	Emotions and intentions	"Ali failed an important exam" → he may feel disappointed

Combining Facts: The Ice Cream Example

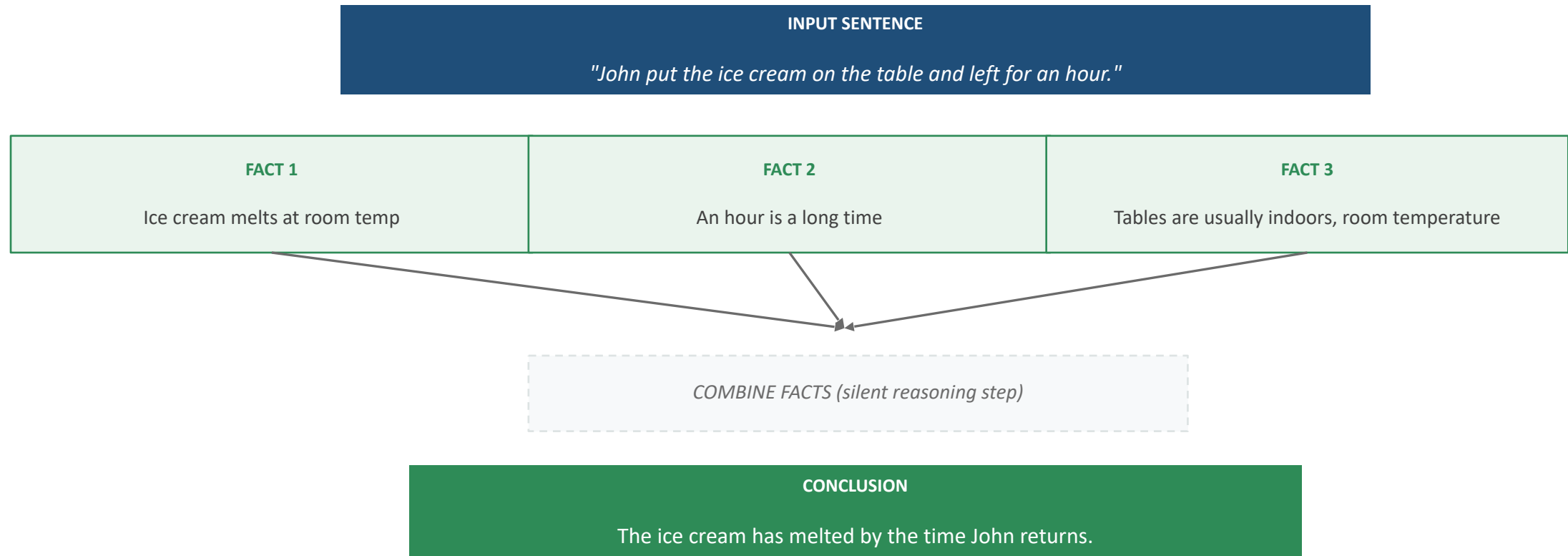


Figure 2.1 — Three independent, unstated commonsense facts combine to reach a conclusion a human draws instinctively.

IMPORTANT PRINCIPLE

Commonsense knowledge is compositional — real understanding rarely comes from one fact alone, but from quietly combining several small, "obvious" facts at once.

The Frame Problem

One of the most famous problems in AI, introduced in the late 1960s. Imagine a robot in a room that performs the action `Move(Box)`. Obviously, the box's location changed. But what about everything else? The wall color, floor, ceiling, lights, and windows all remain unchanged — and there are thousands of such facts.

DEFINITION

The Frame Problem asks: how can an AI efficiently represent the effects of an action without explicitly listing everything that remains unchanged?

Without solving this problem, every single action a robot takes would require updating millions of irrelevant facts, making the system computationally infeasible. Modern solutions (such as the Event Calculus in Topic 3) assume that, by default, everything stays the same ***unless*** an explicit rule says otherwise — directly reusing the default-reasoning idea from Topic 1.

The Qualification Problem

Consider the simple rule: "Turning the key starts the car." Sounds simple — but consider the exceptions: dead battery, no fuel, engine failure, wrong key, damaged ignition. There are potentially thousands of hidden conditions.

DEFINITION

The Qualification Problem asks: how can we specify all the conditions required for an action to succeed? Every everyday rule (like "birds fly") carries hidden assumptions (healthy, has wings, not trapped, not injured) that humans ignore naturally but that AI systems must somehow handle.

This is closely related to the Frame Problem: where the Frame Problem asks what stays the same, the Qualification Problem asks what could silently prevent an action from working at all.

The Yale Shooting Problem

A famous benchmark that demonstrates how reasoning about actions and time can produce unintuitive results if not handled carefully.

Initial state: Gun is loaded. Person is alive.

Actions (in order): Load Gun → Wait → Shoot

Expected conclusion: Person is dead.

This seems obvious to any human. Yet early logical systems sometimes incorrectly concluded that the person remained alive, because the formal system failed to correctly handle **persistence** (the gun staying loaded through the "Wait" action) and the precise effects of actions over time. This failure directly motivated the more careful temporal frameworks covered in Topic 3, including the Event Calculus.

WHY THIS PROBLEM MATTERS

The Yale Shooting Problem proved that time, actions, and persistence all matter for correct reasoning — a fact that remains true must be assumed to persist through irrelevant actions (like "Wait") unless something explicitly changes it.

Commonsense Knowledge Bases

Since hand-coding commonsense at scale is so difficult, researchers built dedicated projects to store it explicitly:

(a) Cyc Developed by Douglas Lenat, aiming to build a database of millions of facts like "humans need food." Strength: rich, structured reasoning. Weakness: extremely expensive and slow to build.

(b) ConceptNet A large commonsense knowledge graph from MIT Media Lab, storing relations like Car → UsedFor → Transportation, letting AI infer that a restaurant visitor likely wants food.

(c) ATOMIC A modern knowledge base focused on events and social reasoning. For "Ali helps an elderly person cross the road," it stores: Ali is kind, the person feels grateful, Ali intended to help.

Knowledge Base	Focus	Built By
Cyc	General logical commonsense facts	Douglas Lenat / Cycorp
ConceptNet	Everyday concept relationships	MIT Media Lab
ATOMIC	Events, social causes and effects	Allen Institute for AI

Commonsense Reasoning in Large Language Models

Modern systems like GPT appear to possess commonsense — not because anyone hand-coded rules, but because they learned from billions of examples in books, articles, and websites during training. Patterns become embedded directly in the network's weights.

ILLUSTRATION — Does an LLM "Know" Ice Cream Melts?

Prompt: "John put ice cream on the dashboard of his car on a hot day. What happened later?"

1. A modern LLM will typically answer "the ice cream melted," because it has learned strong statistical associations between heat and melting from its training data.
2. This raises an open research question: does the model truly understand causation, or has it only learned a powerful statistical shortcut?

OPEN RESEARCH DEBATE

Some researchers argue LLMs possess genuine statistical commonsense — broad, useful, but occasionally brittle. Others argue true understanding requires explicit symbolic reasoning layered on top. This exact debate motivates Topic 5: Neuro-Symbolic AI, which tries to combine both.

The Winograd Schema Challenge

PROBLEM

Consider this sentence: "The city councilmen refused the demonstrators a permit because they feared violence."

Question: Who does "they" refer to — the councilmen or the demonstrators? Explain how commonsense reasoning answers this, since grammar alone cannot.

PAUSE HERE — work out your answer before the next slide

The Winograd Schema Challenge — Step-by-Step

- 1 Check what grammar alone can tell us: "they" could grammatically refer to either group — both are plural nouns earlier in the sentence. Grammar is insufficient to resolve this on its own.
- 2 Bring in commonsense fact: people in positions of authority (councilmen) who deny a permit typically do so out of caution about safety risks they perceive.
- 3 Bring in a second commonsense fact: it would be unusual for demonstrators to fear violence from themselves — fearing your own actions doesn't match how intentions normally work.
- 4 Combine both facts: the more sensible interpretation is that "they" = the councilmen, since it is their fear of potential violence that explains why they refused the permit.

WHY THIS SOLUTION WORKS

This is a famous test specifically designed so that swapping one word (changing "feared violence" to "advocated violence") flips the correct answer to the other noun. This proves the sentence cannot be resolved by grammar or word-order statistics alone — it requires genuine reasoning about plausible human motives.

Object Permanence and Physical Commonsense

PROBLEM

A robot's camera sees a ball roll behind a couch and disappear from view. Three seconds later, the robot is asked: "Where is the ball?"

Question: *What is the commonsense-correct answer, and what reasoning must the robot perform to avoid a wrong answer?*

PAUSE HERE — *work out your answer before the next slide*

Object Permanence and Physical Commonsense

- 1 A naive vision-only system might answer "The ball does not exist" or "unknown," because it can no longer see the ball in any camera frame.
- 2 Apply the commonsense principle of object permanence: physical objects continue to exist even when not currently visible — they do not vanish from reality just because they leave a camera's view.
- 3 Apply a second commonsense fact: solid objects cannot pass through other solid objects (the couch). So the ball must still be somewhere behind or near the couch.
- 4 Combine both facts: "The ball is most likely still behind the couch, since nothing was observed to move it elsewhere."

WHY THIS SOLUTION WORKS

This is "naive physics" commonsense reasoning — the same kind a human infant develops around 8 months of age, before learning language. For self-driving cars, a pedestrian walking behind a parked van must still be assumed to exist, rather than assuming the road ahead is clear.

Script-Based Reasoning — The Restaurant Script

PROBLEM

You are told only this: "Maria went to a restaurant. She ordered the salmon. She left a big tip."

Question: *Using script-based commonsense reasoning, what unstated events almost certainly also happened, and how does the system infer them?*

PAUSE HERE — *work out your answer before the next slide*

Script-Based Reasoning — The Restaurant Script

- 1 Recognize the situation as matching a known script: the "going to a restaurant" script — enter → get seated → receive a menu → order → food prepared → served → eat → pay (often with a tip) → leave.
- 2 Match stated events to slots in the script: "ordered the salmon" fits the "order" slot; "left a big tip" fits near the end of the "pay" slot.
- 3 Fill in the unstated slots: the system infers Maria was seated, given a menu, and the salmon was prepared and served — even though none of this was explicitly stated.
- 4 Infer a secondary, weaker conclusion: a "big tip" is commonly associated with satisfaction, so the system can tentatively guess Maria enjoyed her meal — though this is more tentative than the other inferences.

WHY THIS SOLUTION WORKS

Scripts let an AI fill enormous gaps using a small number of stated facts. Notice the link to Topic 1: "Maria was satisfied" is a default, not a certainty — if the next sentence said she tipped out of pity for the waiter, that conclusion would be instantly retracted. Commonsense and non-monotonic reasoning work hand-in-hand.

TOPIC 03

Temporal Reasoning and Event Representation

Teaching AI to understand "before," "after," "while," and "how long."

Why Do We Need Temporal Reasoning?

Consider a simple statement: "Ali is in Lahore." Is this always true? No — Ali may be in Lahore today, Islamabad tomorrow, and Dubai next week. The truth of the statement **changes with time**. Traditional logic represents facts as static truths, such as `InCity(Ali, Lahore)` — but this representation quietly ignores one critical question: **when?** Without time information, the knowledge is incomplete.

REAL-WORLD MOTIVATION: A HOSPITAL AI

At 8:00 AM, a patient is Stable. At 10:00 AM, the patient becomes Critical. At 1:00 PM, the patient is Recovering. If the AI ignores time, all three statements appear simultaneously true — a direct contradiction. Therefore, AI must explicitly represent when each fact holds.

Most real-world AI systems must deal primarily with dynamic knowledge — facts whose truth changes over time, as shown on the next slide.

Static vs. Dynamic Knowledge

Static Knowledge	Dynamic Knowledge
2 + 2 = 4	Ali is in Lahore
Water boils at 100°C	Temperature is 35°C
Earth revolves around the Sun	Traffic congestion is high
Rarely changes	Continuously changes
Application Areas Where Time Matters Most	

Application	Why Time Matters
Robotics	Actions must occur in the correct sequence
Healthcare	Symptoms evolve over time
Autonomous Vehicles	Decisions must be made in milliseconds
Finance / Security	Markets and event sequences change moment to moment

What Is Temporal Reasoning?

DEFINITION

Temporal reasoning is the ability of an AI system to represent, understand, and reason about time-dependent information — answering questions like "What happened first?", "How long did something last?", "Did two events overlap?", and "What state existed before/after an event?"

Humans perform temporal reasoning constantly and effortlessly. Told "*Ali graduated in 2024*" and "*Ali started a job in 2025*," we instantly infer graduation happened first — no visible reasoning process is needed. AI, however, must do this **explicitly**.

Representing Time: Points vs. Intervals

Point-Based Representation	Interval-Based Representation
Treats time as individual instants, like ticks on a clock (8:00 AM, 9:00 AM, 10:00 AM...). Simple and works well for timestamps, but becomes inefficient for long activities — representing "Ali studied from 8 PM to 11 PM" minute-by-minute would be wasteful.	Treats an entire duration — a meeting, a class, a surgery, an exam — as a single object with a start and an end. Far more natural for real-world activities, which almost always have duration rather than being instantaneous.

Concept	What It Captures	Typical Use
Point-based time	Instant moments only	Simple event ordering, timestamps
Interval-based time (Allen's Algebra)	Durations and overlaps	Scheduling, calendars, narratives

All Thirteen Relations

Introduced by **James F. Allen in 1983**, this is one of the most famous frameworks in temporal reasoning. Given two events (intervals) A and B, Allen showed there are exactly thirteen possible relationships between them — not just "before" and "after."

Relation	Meaning	Example
Before / After	One interval ends before the other starts	Lecture before Exam
Meets / Met-by	One ends exactly when the other begins (no gap)	Lecture ends exactly when Exam begins
Overlaps / Overlapped-by	They share some time but not all	Two university classes sharing some time
Starts / Started-by	Same start point, different end points	Two runners starting a race together
Finishes / Finished-by	Same end point, different start points	Two flights landing at the same time
During / Contains	One interval is entirely inside the other	Midterm Exam during the Semester
Equals	Both intervals are identical	Two clocks showing the same meeting time

Counting each direction separately (Before/After, Meets/Met-by, etc.) gives exactly 13 relations in total — the complete set Allen identified.

Four of Allen's Interval Relations

time →

BEFORE



OVERLAPS



DURING



MEETS



Figure 3.1 — Four of Allen's interval relations: Before, Overlaps, During, and Meets, shown as time-proportional bars.

IMPORTANT PRINCIPLE

Order in a story is not order in time. Natural language often describes events out of chronological sequence (flashbacks, explanations, "by the way" remarks). A temporal reasoning system must reconstruct the true timeline.

Events and Event Representation

DEFINITION

An event is an occurrence that causes a change in the state of a system. Examples: OpenDoor, TurnOnLight, StartEngine, BookTicket, PayBill.

Every event typically has four components: **an event name** (e.g., OpenDoor), **a time** (e.g., 10:00 AM), **participants** (e.g., Ali, Door), and **an effect** (e.g., door becomes open).

Event Name

Time

Participants

Effect

States and Fluents

A **state** describes the condition of the world at a particular time (e.g., "Door Open," "Room Occupied"). A **fluent** is a property whose value can change over time — such as DoorOpen, Temperature, Location(Ali), or BatteryLevel.

```
At 8:00 AM:   DoorOpen = False
Event:        OpenDoor
At 8:01 AM:   DoorOpen = True    ← the fluent changed
```

Without fluents, an AI system cannot track changing states — they are fundamental in robotics, smart homes, healthcare monitoring, and autonomous vehicles.

Situation Calculus

One of the earliest AI reasoning systems for representing actions and their effects. The core idea: represent the world as a sequence of *situations*, where each action transforms one situation into the next.

```
Situation S0:  Door Closed
      ↓ Action: OpenDoor
Situation S1:  Door Open
```

Example — robot navigation: in situation Robot(RoomA), the action Move(RoomB) produces the result Robot(RoomB). Situation Calculus models these transitions cleanly, but has a major limitation: large systems generate huge numbers of situations, becoming computationally expensive to track.

Event Calculus

Developed as an improvement over Situation Calculus, and now widely used in temporal reasoning. Instead of focusing on situations, it focuses on **events and their effects on fluents over time** — combining the event idea from Section 3.7 with the fluent idea from Section 3.8.

```
Initially:      DoorOpen = False  
Event at 10 AM: OpenDoor  
After:         DoorOpen = True
```

Event Calculus is widely used in smart buildings, healthcare monitoring, security systems, and robotics — anywhere events naturally occur and accumulate over time.

Temporal Reasoning in Robotics, Healthcare & Smart Cities

Robotics

Robots continuously reason about correct order. A warehouse robot with tasks Pick → Move → Deliver cannot deliver before picking up. An autonomous parking robot must Lift, then Move, then Place, in that exact order.

Healthcare

Patient timeline: 8:00 AM Fever → 10:00 AM Medication → 12:00 PM Temperature Reduced. The AI infers the medication likely caused the improvement. The sequence Irregular heartbeat → BP drop → Cardiac arrest helps predict emergencies before they fully develop.

Smart Cities

Traffic systems track patterns like 8 AM (Heavy) → 10 AM (Moderate) → 1 PM (Low) to predict future conditions and optimize signal timing or rerouting in real time.

IMPORTANT PRINCIPLE

Across robotics, healthcare, and smart cities, the same underlying tools apply: events, fluents, and interval relationships. Once a domain is modeled this way, the same reasoning machinery can answer very different real-world questions.

Reconstructing a Timeline from a Story

PROBLEM

Read this short passage: "Hamid submitted his thesis on Monday. Before that, his supervisor had reviewed three full drafts. He had started writing the thesis two months earlier."

Question: *Place these three events in correct chronological order using temporal reasoning, even though the text does not list them in time order.*

PAUSE HERE — work out your answer before the next slide

Reconstructing a Timeline from a Story

- 1 Extract the three events: (E1) Hamid started writing. (E2) The supervisor reviewed three drafts. (E3) Hamid submitted the thesis on Monday.
- 2 Extract the explicit temporal clues: "Before that" links E2 before E3. "Two months earlier" links E1 before E2 (drafts must exist before being reviewed).
- 3 Apply transitivity, a core rule of temporal logic: if E1 is before E2, and E2 is before E3, then E1 must also be before E3 (like $A < B$ and $B < C$ implies $A < C$).
- 4 Final chronological order: E1 (started writing) → E2 (supervisor reviewed drafts) → E3 (submitted thesis on Monday).

WHY THIS SOLUTION WORKS

The sentence order in the passage was E3, E2, E1 — the exact reverse of the true timeline. This is extremely common in natural writing. A temporal reasoning system must extract relational clues ("before that," "earlier") rather than relying on sentence position, then apply logical transitivity to merge pairwise relationships into one complete timeline.

Applying Allen's Interval Algebra

PROBLEM

A factory's machine maintenance window runs from 2:00 PM to 4:00 PM. A power outage is reported from 3:00 PM to 3:30 PM.

Question: *Using Allen's Interval Algebra, what is the precise relationship between the maintenance window and the power outage, and what does this tell an automated scheduling system?*

PAUSE HERE — work out your answer before the next slide

Applying Allen's Interval Algebra

- 1 Represent both events as intervals: Maintenance = [2:00 PM, 4:00 PM]. Outage = [3:00 PM, 3:30 PM].
- 2 Compare start times: the outage starts (3:00 PM) after maintenance starts (2:00 PM). Compare end times: the outage ends (3:30 PM) before maintenance ends (4:00 PM).
- 3 Match this pattern to Allen's relation set: when one interval's start and end both fall strictly inside another's, the relation is "During" — the outage occurs during the maintenance window.
- 4 Practical conclusion: no conflict needs flagging for separate rescheduling, because the entire outage is already contained within the planned maintenance downtime.

WHY THIS SOLUTION WORKS

This demonstrates why interval algebra is more useful than simple before/after reasoning for real systems like factory scheduling or hospital bed bookings. If the system only tracked single points in time, it might incorrectly flag the outage as a new disruption. Recognizing the precise "During" relationship shows the outage was already absorbed by planned downtime.

Event Calculus — Tracking a Changing Fluent

PROBLEM

Consider this sequence given to a smart-home AI system:

Event at 6:00 PM: "Ali turns on the heater."

Event at 6:45 PM: "Ali opens the window."

Event at 7:10 PM: "Ali turns off the heater."

Question: *Using Event Calculus thinking, what is the state of "heater status" at 7:00 PM, and how is this determined?*

PAUSE HERE — *work out your answer before the next slide*

Event Calculus — Tracking a Changing Fluent

- 1 Identify the relevant fluent: "heater is on" (true or false).
- 2 Identify which events initiate or terminate this fluent: "turns on the heater" initiates it; "turns off the heater" terminates it. "Opens the window" does not affect this fluent at all.
- 3 Build the fluent's timeline: "heater is on" becomes true at 6:00 PM and remains true continuously until 7:10 PM — nothing in between changed it back.
- 4 Check the query time (7:00 PM) against this timeline: it falls between 6:00 PM and 7:10 PM, so the fluent is still true. Conclusion: the heater was on at 7:00 PM.

WHY THIS SOLUTION WORKS

Event Calculus lets the system answer questions about any moment — including 7:00 PM, never directly mentioned — by reasoning about which events changed which facts, and when. The window-opening event was correctly recognized as irrelevant to the heater fluent. This selective tracking is exactly what lets smart-home systems and robotics controllers maintain an accurate picture of the world from a sparse list of events.

TOPIC 04

Knowledge Graphs and Real-World Applications

Connecting facts as networks — the structure behind modern search and AI assistants.

Why Knowledge Graphs? A Simple Chain of Questions

Suppose I ask: "Where is UMT located?" You answer: "UMT is in Lahore." I ask: "Where is Lahore located?" You answer: "Lahore is in Pakistan." I ask: "Which continent contains Pakistan?" You answer: "Asia."

UMT → Lahore → Pakistan → Asia, so UMT is in Asia

You can now immediately infer this **even though nobody ever told you that directly**. You inferred it by chaining relationships. Knowledge graphs are designed to support exactly this kind of reasoning.

The Problem with Traditional Databases

The same information could live in three separate tables — Universities, Cities, Countries — with UMT linked to Lahore, Lahore linked to Pakistan, and Pakistan linked to Asia. The information technically exists, but discovering the indirect relationship requires writing a multi-step join query.

Humans naturally think in relationships; traditional databases store isolated records, while knowledge graphs store connections as first-class objects.

4.3 The Evolution of Knowledge Representation

Stage	Idea	Limitation
1. Rule-Based Systems	IF Bird(X) THEN Fly(X)	Difficult to scale
2. Semantic Networks	Nodes connected by simple links (Bird → Animal)	Limited expressiveness
3. Ontologies	Formal meanings and rules added on top	Still mostly manual
4. Knowledge Graphs	Graphs + ontologies + semantics + ML	Foundation of modern search/AI

What Is a Knowledge Graph?

DEFINITION

A knowledge graph is a structured representation of knowledge where entities (objects, people, places, organizations, concepts) are connected through relationships, optionally annotated with attributes (properties describing an entity, such as Name, Age, or CGPA).

In graph terms: entities are **nodes** and relationships are **edges**. A small graph — Ali → StudiesAt → UMT → LocatedIn → Lahore → LocatedIn → Pakistan — lets the system infer "Ali is associated with Pakistan" even though that fact was never explicitly stored.

Many cognitive scientists believe the human brain organizes knowledge similarly — hearing "Lahore" instantly activates related concepts like Pakistan, Punjab, food, history, and universities, much like a knowledge graph's associative structure.

Semantic Networks and Inheritance

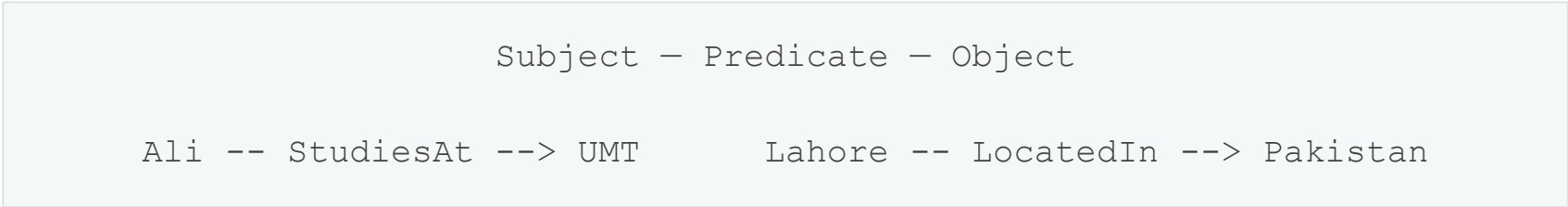
Knowledge graphs evolved from *semantic networks*, which represent concepts and relationships such as Sparrow IS-A Bird and Bird IS-A Animal.

```
Animal → Can Move  
Bird → Has Wings  
Sparrow IS-A Bird IS-A Animal
```

One major advantage is *inheritance*: the system automatically infers Sparrow Can Move and Sparrow Has Wings — without ever storing those facts directly for "Sparrow." This dramatically simplifies large knowledge representations.

Every Fact Becomes a Triple

Every single fact in a knowledge graph is broken into the same simple three-part structure, called the **Resource Description Framework (RDF) triple**:



Subject	Predicate (Relationship)	Object
Marie Curie	wonAward	Nobel Prize in Physics
Marie Curie	bornIn	Warsaw
Nobel Prize in Physics	awardedIn	1903

By breaking every fact into this tiny, uniform shape, millions (or billions) of facts from completely different domains can be stored, linked, and queried using exactly the same underlying structure.

Ontologies

Students often confuse ontologies with knowledge graphs. Think of an **ontology** as the grammar of knowledge — it defines the vocabulary, classes, and rules — while the knowledge graph itself is the actual knowledge built using that grammar.

EXAMPLE: A UNIVERSITY ONTOLOGY

Classes: Student, Professor, Course, Department, University.

Relationships: StudiesAt, Teaches, Offers, BelongsTo.

Rules: Every Student studies a Course; every Professor teaches a Course.

Without an ontology, "Student," "Learner," and "Pupil" might be treated as unrelated concepts. The ontology provides consistency — the same role medical ontologies play for Disease, Patient, Medicine, and Doctor in healthcare AI systems.

A Small Knowledge Graph

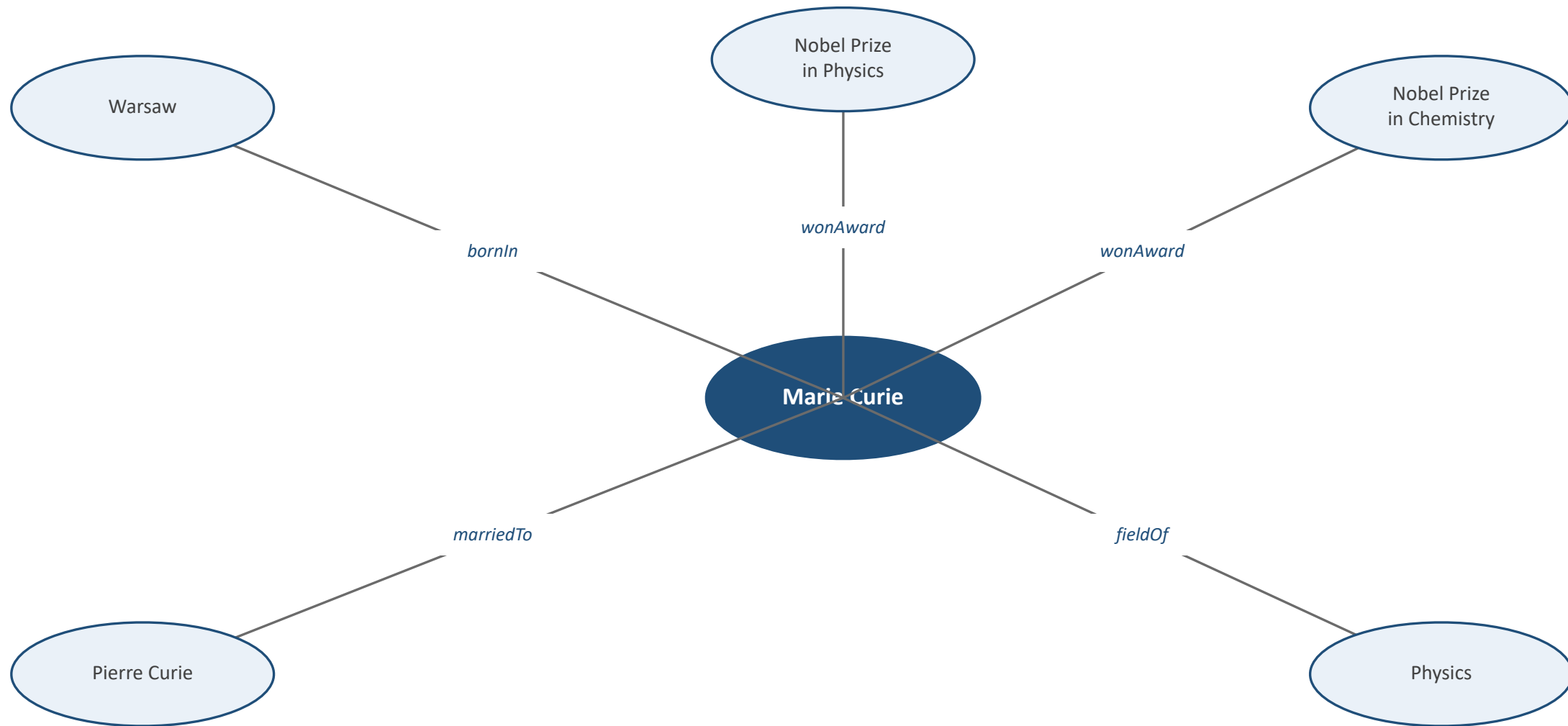


Figure 4.1 — Every line is a triple: (Marie Curie, bornIn, Warsaw), (Marie Curie, wonAward, Nobel Prize in Physics), and so on.

Direct vs. Inferred Knowledge

Direct knowledge is stored explicitly (e.g., "UMT LocatedIn Lahore"). **Inferred knowledge** is generated automatically through reasoning — most commonly via ***transitive reasoning***: if $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$. This is exactly the UMT $\rightarrow$ Lahore $\rightarrow$ Pakistan $\rightarrow$ Asia chain from Section 4.1, now formalized.

IMPORTANT PRINCIPLE

Answering a question = walking a path. "What field did Marie Curie's first Nobel Prize belong to?" is answered by simply following the edges: Marie Curie $\rightarrow$ wonAward $\rightarrow$ Nobel Prize in Physics $\rightarrow$ fieldOf $\rightarrow$ Physics. No separate "search logic" is needed.

Knowledge Graph Search and Traversal

Suppose we want: "Find all universities in Pakistan." The graph traverses University → LocatedIn → Pakistan directly. Or: "Find students supervised by Professor X" follows Professor → Supervises → Student. Graph traversal makes this kind of discovery simple and efficient, without scanning unrelated records.

4.11 Knowledge Graph Embeddings

DEFINITION

A knowledge graph embedding converts each entity and relationship into a numerical vector (e.g., Ali = [0.3, 0.8, 0.2], UMT = [0.4, 0.7, 0.1]), so machine learning algorithms can process the graph for tasks like link prediction, recommendation, classification, and similarity search.

For example, if Ali and Ahmed both StudiesAt UMT, an embedding algorithm might learn that their vectors are close together, allowing the system to predict *"Ali and Ahmed likely share interests"* — a pattern never explicitly stored as a fact.

Knowledge Graphs and Large Language Models

LLMs can **hallucinate** — forgetting facts or generating outdated or fabricated information — because they predict probable words rather than verifying facts. Knowledge graphs provide verified facts; LLMs provide natural language understanding. Combined, they produce AI that is both accurate and fluent.

WHY THIS COMBINATION MATTERS

A pure LLM might confidently invent a fake statistic. A knowledge graph alone cannot hold a fluent conversation. Pairing them — LLM for language, knowledge graph for facts — is one of the most actively researched directions in making AI systems more trustworthy.

This idea is the direct bridge into Topic 5: Neuro-Symbolic AI.

Knowledge Graphs Powering Everyday AI

- (a) Search Engines** Searching "Cristiano Ronaldo" instantly shows age, teams, awards, family, statistics — pulled from Google's knowledge graph, which searches relationships rather than individual web pages.
- (b) Recommendation Systems** If User A bought a Laptop, and the graph knows Laptop is associated with Mouse, and Mouse with Keyboard, the system infers: recommend Mouse and Keyboard.
- (c) Social Networks** If Ali knows Ahmed, and Ahmed knows Sara, the graph can infer "Ali may know Sara" — the basis of friend-recommendation features on LinkedIn and Facebook.
- (d) Healthcare** Medical knowledge graphs connect Symptoms, Diseases, Treatments, Medications — e.g., Fever → Infection → Antibiotics — helping AI assist doctors in diagnosis.

Application	Core Relationship Used
Google Search	Person → hasAward / hasTeam / hasStat
Amazon Recommendations	Product → associatedWith → Product
LinkedIn / Facebook	Person → knows → Person
Healthcare AI	Symptom → indicates → Disease → treatedBy → Medicine

Building Triples from a Sentence

PROBLEM

Convert this sentence into knowledge-graph triples: "Steve Jobs co-founded Apple in Cupertino in 1976."

Question: *What set of (subject, predicate, object) triples correctly captures every fact in this sentence?*

PAUSE HERE — *work out your answer before the next slide*

Building Triples from a Sentence

- 1 Identify all the distinct entities mentioned: "Steve Jobs," "Apple," "Cupertino," and "1976."
- 2 Identify the relationships connecting them: founding, location of founding, and year of founding are three separate relationships, even though they appear in one sentence.
- 3 Write each relationship as its own triple: (Steve Jobs, co-founded, Apple) · (Apple, foundedIn, Cupertino) · (Apple, foundedYear, 1976).
- 4 Check completeness: every noun phrase from the original sentence appears in at least one triple, and no information has been lost or duplicated.

WHY THIS SOLUTION WORKS

A single sentence often bundles several independent facts. Breaking it into separate triples is essential because each fact may need to be queried, updated, or corrected on its own — if a historian later determines Apple was founded in a garage in Los Altos rather than Cupertino, only that one triple needs correcting, leaving the founding year and co-founder fact untouched.

Multi-Hop Reasoning Across a Graph

PROBLEM

A knowledge graph contains these triples:

```
(Ahmed, worksAt, TechCorp) · (TechCorp, locatedIn, Lahore) · (Lahore, locatedIn, Pakistan)
```

Question: Using only these triples, what country does Ahmed work in? Show the multi-hop reasoning path.

PAUSE HERE — work out your answer before the next slide

Multi-Hop Reasoning Across a Graph

- 1 Notice that no triple directly states "Ahmed worksIn Pakistan." The answer must be derived, not looked up directly — this is called multi-hop reasoning.
- 2 First hop: start at "Ahmed," follow the worksAt edge → arrive at "TechCorp."
- 3 Second hop: from "TechCorp," follow the locatedIn edge → arrive at "Lahore."
- 4 Third hop: from "Lahore," follow another locatedIn edge → arrive at "Pakistan." Conclusion: Ahmed works in Pakistan, reached via a three-hop path.

WHY THIS SOLUTION WORKS

This is exactly how modern search engines and virtual assistants answer questions never explicitly written down anywhere. The number of hops matters in practice: more hops generally mean a less certain answer, which is why many real-world systems report a confidence score that decreases slightly with each additional hop.

Using an Ontology to Catch a Contradiction

PROBLEM

An ontology defines the rule: "Only entities of class Person can have the relationship marriedTo. A City cannot be marriedTo anything."

A data-entry error accidentally adds the triple:

(Lahore, marriedTo, Karachi).

Question: *How does the ontology allow the system to detect this as an error automatically, without a human reviewing every triple by hand?*

PAUSE HERE — work out your answer before the next slide

Using an Ontology to Catch a Contradiction

- 1 Look up the classes of the entities involved: "Lahore" and "Karachi" are both classified under the ontology as instances of the class City.
- 2 Look up the ontology's domain/range constraint for marriedTo: it specifies that both the subject and object must belong to the class Person.
- 3 Compare the actual triple against the rule: the triple uses class City on both sides, but the rule requires class Person — a direct mismatch.
- 4 The system automatically flags the triple as an ontology violation and either rejects it or sends it for human review — without manually inspecting millions of other triples.

WHY THIS SOLUTION WORKS

This transforms data validation from a manual, human task into an automatic, logical check that scales to billions of facts. Without the ontology, a knowledge graph is just a pile of triples with no way to tell a correct fact from an absurd one — closely related to the commonsense reasoning of Topic 2, but applied to the graph's own data.

TOPIC 05

Neuro-Symbolic AI

Combining Neural Networks and Symbolic Reasoning — the best of pattern-recognition and logic.

The AI Dream and Its Historical Evolution

Since the 1950s, AI researchers have asked: how can we build machines that think like humans? Researchers first believed intelligence could be created simply by encoding logical rules — giving rise to Symbolic AI. Later, researchers realized humans learn from experience rather than memorized rules, leading to Machine Learning and eventually Deep Learning. Today: symbolic AI can reason but struggles to learn; neural networks can learn but struggle to reason.

Phase	Era	Core Idea
1. Symbolic AI	1950–1985	Encode intelligence as logic and rules (Expert Systems)
2. Machine Learning	1980–2010	Teach machines to learn patterns from data
3. Deep Learning Revolution	2012–Present	Deep neural networks outperform humans on perception tasks

THE MODERN PUZZLE

"Ali is older than Ahmed. Ahmed is older than Bilal. Who is oldest?" Humans solve this instantly. Neural networks often need many training examples before reliably learning such reasoning patterns — because they learn statistical patterns rather than explicit logic.

MYCIN and the Knowledge Acquisition Bottleneck

A famous example from Phase 1 is **MYCIN**, developed in the 1970s to diagnose bacterial infections using manually-encoded rules such as: "IF patient has fever AND blood culture positive THEN recommend antibiotic."

MYCIN was remarkably successful, but revealed a serious problem: to build large intelligent systems, experts had to manually write thousands of rules (Bird → Fly, Penguin → Not Fly, Injured Bird → Not Fly, Baby Bird → May Not Fly...), and the number of exceptions became enormous. This **Knowledge Acquisition Bottleneck** made building large symbolic knowledge bases impractical — the same bottleneck discussed for commonsense knowledge in Topic 2.

This impracticality is precisely what motivated the shift away from purely hand-written rules and toward systems that could learn patterns directly from data — setting the stage for the neural-network era.

Symbolic AI in Depth

Symbolic AI represents knowledge using symbols and logical rules, resembling mathematics: $\text{Human}(\text{Socrates}), \text{Human}(X) \rightarrow \text{Mortal}(X) \rightarrow \text{infer Mortal}(\text{Socrates})$. Every step is explicit.

Advantages	Limitations
Fully explainable — every conclusion traceable	Cannot learn automatically; every rule added manually
Logically consistent and predictable	Difficult to scale to millions of real-world rules
Works well even with small data	Poor at perception (images, video, speech)
Strong at deduction, planning, constraint solving	Brittle — unexpected situations often cause failure

Neural Networks in Depth

A neural network, loosely inspired by biological neurons, learns patterns directly from data rather than from explicit rules — shown thousands of cat images, it learns to detect "cat" without anyone writing a rule for whiskers or fur.

Advantages	Limitations
Learns automatically; no manual rule-writing	"Black box" — decisions often unclear even to designers
Excellent for images, speech, video, language	Weak explainability
Handles noisy, imperfect data well	Weak strict logical reasoning
Scales well with very large datasets	Extremely data-hungry

Side-by-Side Comparison

Feature	Symbolic AI	Neural Networks
Learns from data	No	Yes
Logical reasoning	Strong	Limited
Explainable	Strong	Limited
Image recognition	Poor	Excellent
Knowledge representation	Excellent	Weak
Scalability	Difficult	Excellent

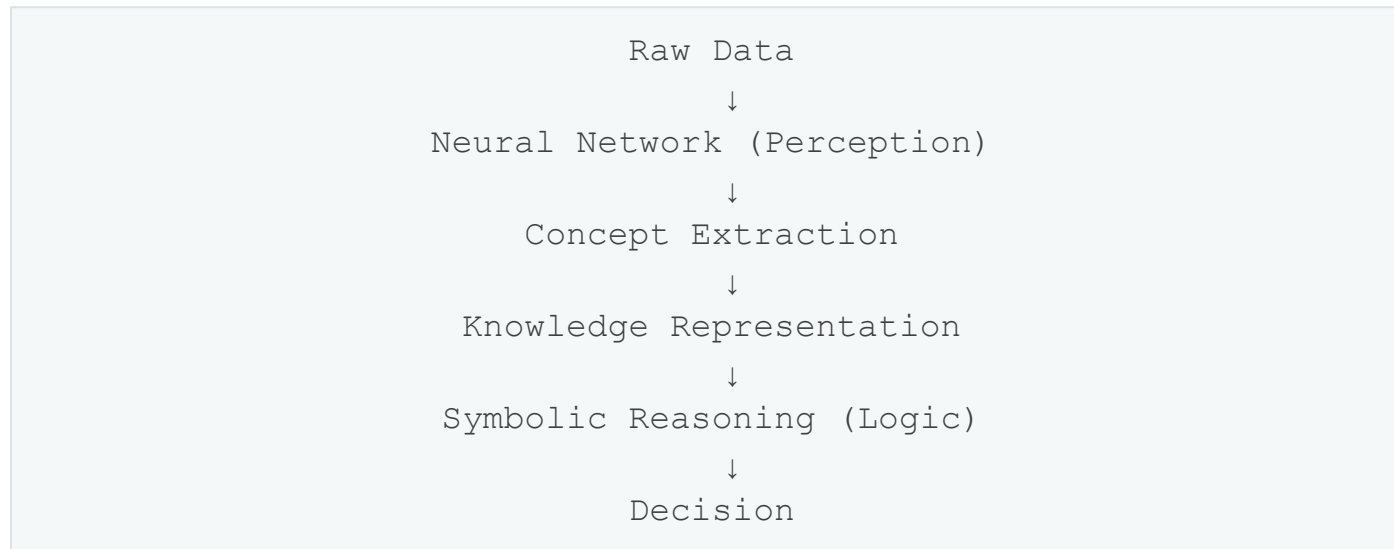
THE CORE INSIGHT

Humans possess both capabilities: we learn (a child learns to recognize a cat) and we reason ("all cats are mammals; Tom is a cat; therefore Tom is a mammal"). Neuro-Symbolic AI attempts to replicate this dual ability in machines.

What Is Neuro-Symbolic AI?

DEFINITION

Neuro-symbolic AI is a hybrid approach that combines the pattern-recognition strength of neural networks with the structured, rule-based reasoning power of symbolic AI, aiming to get the advantages of both while avoiding the weaknesses of either approach alone.



A Neuro-Symbolic Safety System

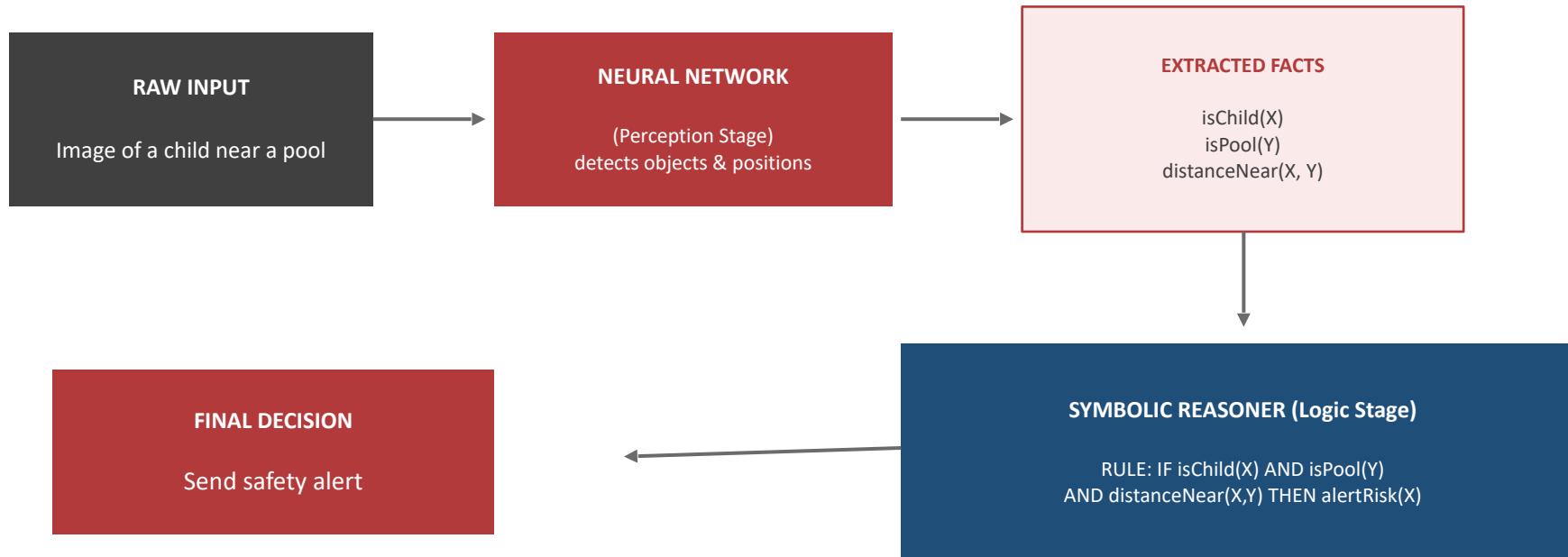


Figure 5.1 — The neural network perceives the scene; the symbolic reasoner applies a clear, explainable rule to decide on an alert.

IMPORTANT PRINCIPLE

Perception and reasoning are different jobs. Asking one giant neural network to both "see" the scene and "reason" about safety rules tends to produce a system that is hard to verify and debug. Splitting the two jobs makes each part simpler and more reliable.

Why Explainability Matters

One of the biggest challenges in modern AI. Suppose an AI rejects a loan application and the customer asks "Why?" A neural network might only offer Confidence = 94% — not helpful. Symbolic reasoning can instead explain: "Income below threshold; credit score insufficient; outstanding debt high." A clear, actionable explanation.

WHY EXPLAINABILITY MATTERS

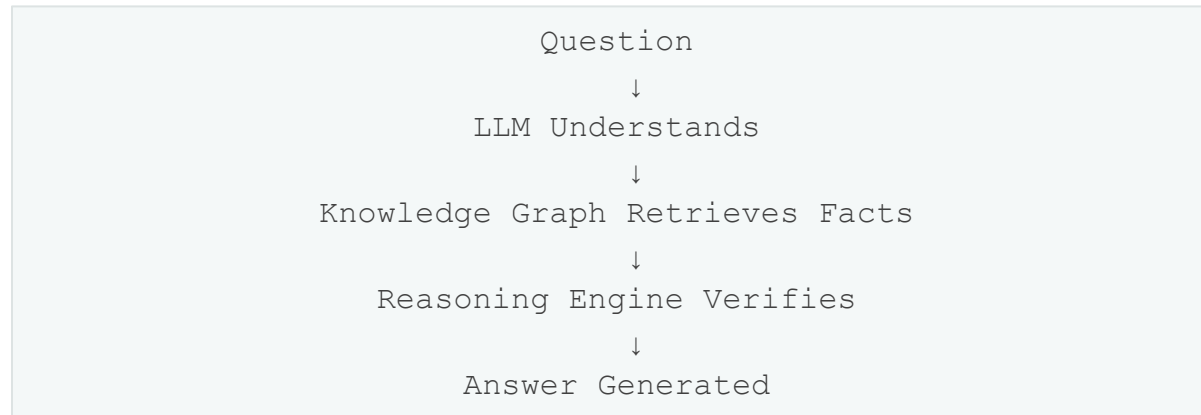
Explainability is critical in healthcare, banking, defense, law, and autonomous vehicles — domains where organizations and regulators increasingly demand explainable decisions, not just accurate ones. Neuro-symbolic systems naturally support this because the final reasoning step is symbolic and traceable.

Knowledge Graphs + Neural Networks

One of the most popular neuro-symbolic approaches: a knowledge graph stores facts such as $\text{Ali} \rightarrow \text{StudiesAt} \rightarrow \text{UMT}$ and $\text{UMT} \rightarrow \text{LocatedIn} \rightarrow \text{Lahore}$. A neural network processes the natural-language question "*Where does Ali study?*", and the knowledge graph supplies the factual answer: "Ali studies at UMT." The knowledge graph provides accuracy; the neural network provides language understanding.

5.10 Large Language Models and Neuro-Symbolic AI

LLMs sometimes hallucinate — inventing references, dates, or statistics — because they predict probable words rather than verifying facts. The neuro-symbolic solution combines an LLM, a knowledge graph, and a logical reasoner:



Neuro-Symbolic AI in Practice

Domain	Neural Component	Symbolic Component
Healthcare	Detect Tumor	Combine with history → Risk Assessment
Robotics	Detect Box	Plan Path, Avoid Collision, Schedule
Autonomous Vehicles	Detect Objects	Apply Traffic Rules & Safety Constraints
Finance	Detect Fraud Pattern	Apply Regulatory Compliance, Explain Decision
Scientific Discovery	Find Patterns in Data	Apply Known Laws to Filter Candidates

LOOKING AHEAD: NEURO-SYMBOLIC AI AND AGI

Many researchers believe neither symbolic AI nor neural networks alone can achieve true AGI, because real intelligence requires both learning and reasoning — the two abilities humans use together effortlessly.

Neuro-Symbolic Visual Question Answering

PROBLEM

A system is shown an image and asked: "Is there a red object that is larger than the blue cube?" The neural network detects:

Object A (red sphere, size = large) · Object B (blue cube, size = small) · Object C (green cylinder, size = medium)

Question: *How does a neuro-symbolic system answer this question correctly, and why is this difficult for a "pure" neural network to do reliably?*

PAUSE HERE — work out your answer before the next slide

Neuro-Symbolic Visual Question Answering

- 1 The neural perception stage outputs structured facts: `color(A, red)`, `shape(A, sphere)`, `size(A, large)`; `color(B, blue)`, `shape(B, cube)`, `size(B, small)`; and so on for Object C.
- 2 The question is parsed into a symbolic logical query: does there exist an object X such that `color(X, red) AND size(X) > size(B)`, where B is the blue cube?
- 3 The symbolic reasoner searches the extracted facts: it finds Object B satisfies "blue cube," then checks all other objects for "red" and "larger than B." Object A matches both conditions.
- 4 Final answer: "Yes" — because Object A (red sphere) is larger than Object B (blue cube). The system can explain its answer by citing exactly these two facts.

WHY THIS SOLUTION WORKS

A "pure" neural network trained end-to-end often struggles because the question requires composing multiple logical conditions — something neural networks can approximate but often get wrong on unusual combinations. By separating perception (network's strength) from logical composition (reasoner's strength), the system handles questions far outside its training examples correctly, and can show its reasoning step by step.

Constraining a Neural Network with Known Rules

PROBLEM

A neural network predicts patient diagnoses from medical scans. On one unusual scan, the raw output assigns: 60% probability "Disease X," and separately, 55% probability "Patient is male AND Disease X requires female anatomy" (a known medical impossibility).

Question: *How does injecting symbolic knowledge correct this contradiction?*

PAUSE HERE — work out your answer before the next slide

Constraining a Neural Network with Known Rules

- 1 Identify the raw, uncorrected output: it has assigned meaningful probability to an outcome that is logically impossible according to established medical fact.
- 2 Introduce the symbolic constraint as a hard logical rule given in advance: "IF patientSex = male, THEN probability(Disease X) MUST equal 0." This comes from established medical knowledge, not the training data.
- 3 Apply the constraint to override the network's raw output: since the rule's condition is true, the impossible probability is forced to zero, regardless of the raw network's calculation.
- 4 The remaining probability is redistributed among medically valid diagnoses, producing a final, logically consistent diagnosis.

WHY THIS SOLUTION WORKS

Neural networks learn purely from statistical patterns in data, and data can contain rare errors, biases, or gaps leading to occasional impossible predictions. By injecting a small set of rock-solid symbolic rules, the system gains a safety net that guarantees certain categories of error cannot occur — a guarantee pure statistical learning alone cannot provide. This matters enormously in healthcare, aviation, and law.

Knowledge-Graph Embeddings for Recommendation

PROBLEM

A streaming platform's knowledge graph contains:

```
(User1, watched, MovieA) · (MovieA, genre, SciFi) · (MovieB, genre, SciFi)
(MovieA, actor, ActorX) · (MovieC, actor, ActorX)
```

Question: Explain, step by step, how a neuro-symbolic recommendation system decides what to recommend to User1, combining the graph's symbolic structure with a neural network's pattern-matching.

PAUSE HERE — work out your answer before the next slide

Knowledge-Graph Embeddings for Recommendation

- 1 The symbolic knowledge graph provides exact, traceable candidate paths: from MovieA, two relationship types lead outward — genre (→ MovieB) and actor (→ MovieC).
- 2 Each entity is converted into a neural embedding — a vector of numbers learned from patterns across millions of other users' behavior, not just this small graph snippet.
- 3 The neural network computes a similarity score between User1's embedding and each candidate, based on subtle patterns the symbolic graph alone cannot represent.
- 4 Suppose MovieC ranks higher. The final recommendation is MovieC, and the system can still explain its choice symbolically: "shares Actor X with a movie you watched."

WHY THIS SOLUTION WORKS

This shows the two components contributing genuinely different value: the symbolic graph generates a small set of plausible, explainable candidates, while the neural network ranks those candidates using subtle statistical patterns no one could hand-write as explicit rules. This is the architecture used by major streaming and e-commerce recommendation systems today.